

Chapter 3
p38 位平面
p39 抖动
p42 颜色查找表

Chapter 4
p54 光谱灵敏度曲线
p55 成像模型
p55 Gamma 校正（原因、做法）
p61 超色域
p62~63 (X,Y,Z)与(R,G,B)的转换
p65~66 不同颜色模型的适用场景
p67 RGB 与 CMY、CMYK 的转换
p68~72 视频中的颜色模型

Chapter 5
p79 数字视频的优点
p80 色度二次采样（为什么要用不同的采样方式，计算不同采样方式的大小）
p81~82 高清电视（基本概念：4K、8K）
p82~85 模拟显示接口和数字显示接口（区别、优势）
p85~89 立体视觉（原理、做法、存在的问题）
p89~90 视差处理 Chapter e
p92~94 声音数字化（原因、过程）
p94~95 Nyquist 理论
p95~97 信噪比、信号量化噪声比（计算）
p97~100 量化、Weber 定律（计算）
p109~118 音频的量化与传输（编码、PCM、无损预测编码、DPCM、DM、ADPDM 的原理、过程）

Chapter 7
p122 压缩率（计算）
p123~125 信息熵（计算） p125 游程编码
p125~126 Shannon-Fano 编码
p127~132 Huffman 编码、扩展 Huffman 编码、自适应 Huffman 编码
p133-136 字典编码-LZW 压缩

一、 图像

图像大小的计算

一幅大小为 M×N、灰度级数为 G 的图像所需的存储空间，即图像的数据量，大小为 **M×N×G（bit）**
直方图的计算

- 1) 计算最大最小值之间的差值 D
- 2) 将差值 D 按要求的坐标轴范围平均分成 N 段，记录每段的上下限
- 3) 统计每段中像素的频度，即为本段直方图纵坐标值

中位切割算法的计算

作用对任意一张图像，使用该算法将图像中的颜色数目减少到 256 色：

- 1) 将图像内的所有像素加入到同一个桶
- 2) 对于该桶中的像素做如下操作：
 - a) 分别计算该桶内的每个颜色分量 (R,G,B) 的最大值与最小值之差，分别设为ΔR, ΔG, ΔB;
 - b) 选出相差最大的那个颜色分量，max(ΔR,ΔG,ΔB)
 - c) 将该分量作为主关键字，去排序此桶内所有像素
 - d) 分割前一半与后一半的像素到二个不同的桶（这里就是"中位切割"名字的由来）
- 3) 重复第二步直到你有 256 个桶
- 4) 将每个桶内的像素颜色平均起来，于是就得到了 256 色

算法的限制
因为每次循环，桶的数量都会加倍，所以最终产生的颜色数量必定为 2 的幂次方。假如有特殊需要减少颜色数目到其它数量时（不是 2 的幂次方，如 12），可以先产生超过需求的颜色数量（如 16），再合并多余的颜色直到符合所需的数量（如 12）。

抖动的计算

用一组二值像素矩阵表示一个灰度像素，即 n*n 的矩阵可以表现 n^2+1 级灰度。例如 2×2 的图案可以表示 5 级灰度
例题：假设一个 5 位的灰度图像，我们需要多大的抖动矩阵来在 1 位的打印机上显示这幅图像？

$$2^5 = 32$$
$$\sqrt{32} \approx 5.66$$

所以需要 6 阶的抖动矩阵

p137~146 **算术编码**（基本算术编码、缩放和增量编码、二进制算术编码、自适应二进制算术编码）
p146~148 无损图像压缩（差分编码、预测器的适用场景）

Chapter 8
p150~151 失真度量
p152 比率失真理论（无损压缩和有损压缩对应曲线上的点）
p152~156 **量化**（均匀标量量化、非均匀标量量化、向量量化）
p156~167 **变换编码**（为什么变换后更适合编码（信息量、冗余度小）、DCT 的过程、DCT 性质（线性变换不损失信息））

Chapter 9
p189~195 **JPEG**(原理、步骤、Zig-zag 扫描的目的) D198-199JPG 2000 的改进
p206 对比 JPEG 与 JPEG2000 在压缩率和压缩效果

Chapter 10
p214 运动估计、**运动补偿**
p214~215 视频压缩的原理
p215~219 搜索运动向量
p220~223 H.261（**帧序列**（1 帧、P 帧，P 帧的编码未必基于运动补偿）、**编解码过程**）
p225~225 H.263（支持半像素精度的预测）

Chapter 11
p231~233 MPEG—1 的运动补偿（帧序列）
p233~235 MPEG-1 与 H.261 的区别

Chapter 20
p462 CB 的定义
p472~473 CBIR 的评价指标(MAP)

二、 音频

音频大小的计算

采样速率×量化位数×声道×时间=存储空间大小（注意单位换算）
采样周期=1/采样速率，成为 1 帧

信号噪声比 SNR

$$SNR = 10 \log_{10} \frac{V_{signal}^2}{V_{noise}^2} = 20 \log_{10} \frac{V_{signal}}{V_{noise}}$$

信号量化噪声比 SQNR

$$SQNR = 20 \log_{10} \frac{V_{signal}}{V_{quan,noise}} = 20 \log_{10} \frac{2^{N-1}}{1} = \mathbf{6.02N(dB)}$$

此处假设了每个采样点的量化精度为 N 位， V_{singal} 取峰值为 2^{N-1} ，同时分母 $V_{quan,singal}$ 取最大值 1/2。注：峰值由采样数/2 得到，即 $2^N \div 2 = 2^{N-1}$

例，假设信号电压 $V_{signal}=0.7V$ ，采样精度用 16bit 表示，求信号量化噪声比。
 $SQNR=6.02*16=96(dB)$

例：假设某声音样本的量化精度为 8 位，则该样本的信噪比为多少分贝？
 $SQNR=6.02*8=48.16(dB)$

预测编码

假设当前样本 $x(n)$ 的预测值 $x_{pre}(n)$ 为前一个样本值，预测误差 $e(n)$ 可写成，

$$x_{pre}(n) = x(n-1)$$
$$e(n) = x(n) - x(n-1)$$

对差值 $e(n)$ 编码比对原始语言样本编码，可用比较少的位数来表达一个样本。

对于具体的语音序列样本，如 $\cdots, x(n-2), x(n-1), x(n), \cdots$ ，如果使用过去几个样本值来预测当前的样本值，合理选择预测系数，产生的预测误差会更小。预测函数可写成，

$$x_{pre}(n) = \sum_{k=1}^N a_{n-k} x(n-k)$$

其中， a_{n-k} 为预测系数； N 为参加预测的样本数，通常 $N=2, 3$ 或 4

【例】假设 $x(n-3)=21$, $x(n-2)=24$, $x(n-1)=27$, $x(n)=23$, 分别求 $N=1$ 和 $N=2$ 的预测误差 $e(n)$ 。

- (1) 当 $N=1$ 时, $e(n)=23-27=-4$ 。
- (2) 当 $N=2$ 时, 为简单起见, 假设预测系数 $a_{n-1}=a_{n-2}=1/2$, 那么 $x_{pre}(n)=a_{n-1}x(n-1)+a_{n-2}x(n-2)=(27+24)/2 \Rightarrow 25$
 $e(n)=x(n)-x_{pre}(n)=23-25=-2$

三、视频

视频存储空间的计算:

一帧图像大小 $\times$ 帧率 $\times$ 时间 (s) (注意单位换算)

场频、帧频、行频

2. 电视扫描术语

- f_f : 场频/场速率(field rate), 每秒钟扫描的场数
 - 根据视觉特性和电网频率(50Hz或60Hz)确定, 使在屏幕上显示的图像看起来不会让人感觉到在闪烁, 以及减低电网频率的干扰
- f_F : 帧频/帧速率(frame rate), 每秒钟扫描的帧数
 - 单位: 帧每秒(frames per second, fps)
 - PAL制和NTSC制的帧频分别为25 fps和30 fps
- f_H : 行频/水平速率(horizontal line rate), 每秒钟扫描的行数
 - 【例】NTSC制精确的帧频是29.97 Hz, 525行每帧, 因此行频为 $29.97 \times 525 = 15734$ 行/秒

2.3 NTSC制的扫描特性

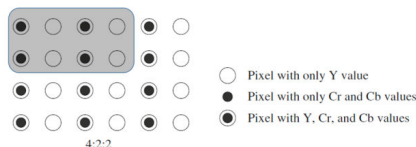
- 一帧图像的总行数为525行, 分两场扫描
- 场扫描频率是60 Hz, 周期为16.67 ms
 - 每场扫描行数: $525/2=262.5$ 行
 - 每场开始保留20条线作为控制信号, 一帧485条线可见
- 帧频(刷新频率)30 Hz(精确29.97), 周期33.33 ms
- 行扫描频率为15 750 Hz, 周期为63.5 μ s, 其中
 - 水平回扫时间为10 μ s(包含5 μ s的水平同步脉冲)
 - 显示图像的时间为53.5 μ s

2.4 SECAM制的扫描特性

- 与PAL制电视的扫描特性类似。

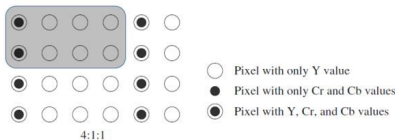
4:2:2

- ◆ “4:2:2”: 平均每个像素需要几个字节?
- ◆ 第一行4个像素: Y=4 Bytes, Cr=Cb=2 Bytes
- ◆ 第二行4个像素采样与第一行相同
- ◆ $(4+2+2) \times 2/8 = 2$ Bytes/pixel



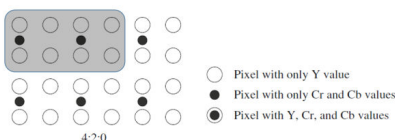
4:1:1

- ◆ “4:1:1”: 平均每个像素需要几个字节?
- ◆ 第一行4个像素: Y=4 Bytes, Cr=Cb=1 Bytes
- ◆ 第二行4个像素采样与第一行相同
- ◆ $(4+1+1) \times 2/8 = 1.5$ Bytes/pixel



4:2:0

- ◆ “4:2:0”: 平均每个像素需要几个字节?
- ◆ 第一行4个像素: Y=4 Bytes, Cr=Cb=2 Bytes
- ◆ 第二行4个像素没有采样色度分量
- ◆ $((4+2+2)+4)/8 = 1.5$ Bytes/pixel



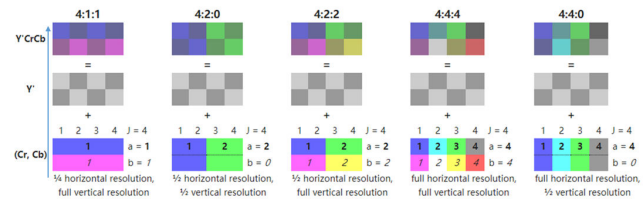
样本数的计算:

认为一个像素包含: Y、Cr、Cb 共 3 个样本, 根据上述情况计算

图像采样-色度的二次采样

J:a:b 表示方法

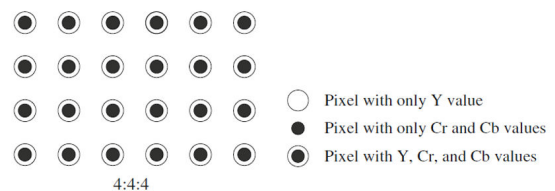
- ◆ J: 水平采样的基准 (默认4个像素)。
- ◆ a: 以2xJ个像素为例, 第一行每J个像素中色度分量 (Cr/Cb) 的采样个数。
- ◆ b: 第二行每J个像素中色度分量的采样个数, 取值为a或者为0。当b=0时, 表示不采样, 直接用第一行的色度值替换
- ◆ 以4:4:4采样为基准



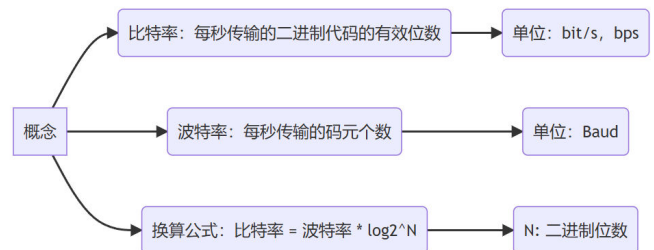
以下例子中, 规定 Y、Cr、Cb 三个分量所占位数相同。

4:4:4

- ◆ “4:4:4”: 视频信号默认情况下, 每个像素 (Pixel) 都有对应的 Y, Cr 和 Cb 值 (每个分量 1 个字节)。
- ◆ 平均每个像素需要3个字节, 存储要求较高
- ◆ 如何在人眼几乎没有察觉的前提下, 节约带宽



比特率的计算:



四、无损压缩

熵的计算

熵代表了信源 X 中每个符号所需要的最小平均位数。

对信源 X 中每个符号进行编码所需的平均位数的下界。

编码方案旨在尽可能地接近熵。

◆ 熵(entropy)

- 按照香农(Shannon)的理论, 在有限的互斥和联合穷举事件的集合中, 熵为事件的信息量的平均值, 也称事件的平均信息量(mean information content), 数学表示为

$$H(X) = \sum_{i=1}^n h(x_i) = \sum_{i=1}^n p(x_i) I(x_i) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i) \quad (3-3)$$

其中, (1) $X = \{x_1, x_2, \dots, x_n\}$ 是事件 $x_i (i=1, 2, \dots, n)$ 的集合, 并满足 $\sum_{i=1}^n p(x_i) = 1$;

(2) $I(x_i) = -\log_2 p(x_i)$ 表示某个事件 x_i 的信息量, 其中 $p(x_i)$ 为事件 x_i 出现的概率, $0 < p(x_i) \leq 1$; $h(x_i) = -p(x_i) \log_2 p(x_i)$ 表示事件 x_i 的熵。

【例】 $X = \{a, b, c\}$ 是由 3 个符号构成的集合, 符号 a, b 和 c 出现的概率分别为 $p(a)=0.5$, $p(b)=0.25$, $p(c)=0.25$, 那么符号 a, b 和 c 的熵分别等于 0.5, 0.5, 0.5, 这个集合的熵为:

$$H(X) = p(a) I(a) + p(b) I(b) + p(c) I(c) = 1.5 \text{ (Sh)}$$

平均位数的计算

压缩比

◆ Compression Ratio

- 设B₀为压缩前表示某信源所需的总位数，B₁为压缩后所需的总位数，则
Compression Ratio (压缩比或压缩率) = B₀ / B₁
- 一般而言，期望压缩比大于1，如果**压缩比越高**，意味着无**损压缩方案越好**

编码效率

◆ 编码效率 η

- η = 原始码长H/为编码后的平均码长L
- 编码效率**越接近于1**，意味着**编码方案越好**

统计编码：给已知统计信息的符号分配代码的数据无损压缩方法
香农-范诺编码

- 在香农的源编码理论中，熵的大小表示非冗余的不可压缩的信息量
- 在计算熵时，如果对数的底数用2，熵的单位就用“香农(Sh)”，也称“位(bit)”。 “位”是1948年Shannon首次使用的术语。例如
事件x_i的熵 $h(x_i) = -p(x_i) \log_2(1/p(x_i))$ ，
表示编码符号x_i所需要的位数

例题：

香农-范诺编码举例

- 有一幅40个像素组成的灰度图像，灰度共有5级，分别用符号A, B, C, D和E表示。40个像素中出现灰度A的像素数有15个，出现灰度B的像素数有7个，出现灰度C的像素数有7个，其余情况见表3-1
- (1) 计算该图像可能获得的压缩比的理论值
- (2) 对5个符号进行编码
- (3) 计算该图像用香农范诺编码后的压缩比的实际值

(3) 算法步骤

- 对于一个给定的符号列表，制定了概率相应的列表或频率计数，使每个符号的相对发生频率是已知。
- 根据频率**降序**排列，频率**高的符号在左边**，频率**低的符号在右边**。
- 分为两部分，使左边部分的总频率和尽可能接近右边部分的总频率和。**使两组差别最小**。
- 该列表的**左半边**分配二进制数字**0**，**右半边**是分配的数字**1**。这意味着，在第一半符号代都是将所有从0开始，第二半的代码都从1开始。
- 对左、右半部分递归应用步骤3和4，细分群体，并添加位的代码，直到每个符号已成为一个相应的代码树的叶。

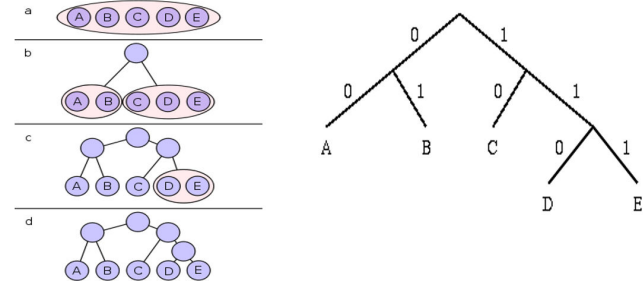


图3-1 香农-范诺算法编码举例

(4) 压缩比的实际值

按照这种方法进行编码需要的总位数为30+14+14+18+15 = 91，实际的压缩比为120:91≈1.32 : 1

表 3-1 符号在图像中出现的数目

符 号	A	B	C	D	E
出现的次数	15	7	7	6	5
出现的概率	$\frac{15}{40}$	$\frac{7}{40}$	$\frac{7}{40}$	$\frac{6}{40}$	$\frac{5}{40}$

(1) 压缩比的理论值

按照常规的编码方法，表示5个符号最少需要3位，如用000表示A，001表示B，...，100表示E，其余3个代码(101, 110, 111)不用。这就意味每个像素用3位，编码这幅图像总共需要120位。按照香农理论，这幅图像的熵为

$$\begin{aligned} H(X) &= -\sum_{i=1}^n p(x_i) \log_2 p(x_i) \\ &= -p(A) \log_2(p(A)) - p(B) \log_2(p(B)) - \dots - p(E) \log_2(p(E)) \\ &= (15/40) \log_2(40/15) + (7/40) \log_2(40/7) + \dots + (5/40) \log_2(40/5) \approx 2.196 \end{aligned}$$

这个数值表明，每个符号不需要用3位构成的代码表示，而用2.196位就可以，因此40个像素只需用87.84位就可以，因此在理论上，这幅图像的的压缩比为120 : 87.84 ≈ 1.37:1，实际上就是3 : 2.196 ≈ 1.37 : 1

(2) 符号编码

对每个符号进行编码时采用“**从上到下**”的方法。首先按照符号出现的频度或概率**降序**排序，如A, B, C, D和E，见3-2。然后使用**递归方法分成两个部分**，每一部分具有近似相同的次数，如图3-1所示

表 3-2 Shannon-Fano 算法举例

符号	出现的次数($p(x_i)$)	$\log_2(1/p(x_i))$	分配的代码	需要的位数
A	15 (0.375)	1.4150	00	30
B	7 (0.175)	2.5145	01	14
C	7 (0.175)	2.5145	10	14
D	6 (0.150)	2.7369	110	18
E	5 (0.125)	3.0000	111	15

霍夫曼编码

◆ 霍夫曼编码举例1

- 现有一个由5个不同符号组成的**30**个符号的字符串：BABACACADADABBCBABEBEDDABEEEBB
- 计算
 - (1) 该字符串的霍夫曼码
 - (2) 该字符串的熵
 - (3) 该字符串的平均码长
 - (4) 编码前后的压缩比

统计概率

符号出现的概率

符号	出现的次数	$\log_2(1/p_i)$	分配的代码	需要的位数
B	10	1.585	?	
A	8	1.907	?	
C	3	3.322	?	
D	4	2.907	?	
E	5	2.585	?	
合计	30			

(1) 计算该字符串的霍夫曼码

- 步骤①：按照符号出现概率大小的顺序对符号进行排序
- 步骤②：把概率最小的两个符号组成一个节点P1
- 步骤③：重复步骤②，得到节点P2, P3, P4,, PN，形成一棵树，其中的PN称为根节点
- 步骤④：从根节点PN开始到每个符号的树叶，从上到下标上0(上枝)和1(下枝)，至于哪个为1哪个为0则无关紧要，但通常把概率大的标成1，概率小的标成0
- 步骤⑤：从根节点PN开始顺着树枝到每个叶子分别写出每个符号的代码

- (1) 计算该字符串的霍夫曼码
- ① 按照符号出现概率大小的顺序对符号进行排序
 - ② 把概率最小的两个符号组成一个节点P1
 - ③ 重复步骤②，得到节点P2, P3, P4,, PN, 形成一棵树，其中的PN称为根节点
 - ④ 从根节点PN开始到每个符号的树叶，从上到下标上0(上枝)和1(下枝)，至于哪个为1哪个为0则无关紧要，但通常把概率大的标成1，概率小的标成0
 - ⑤ 从根节点PN开始顺着树枝到每个叶子分别写出每个符号的代码

5个符号的代码				
符号	出现的次数	$\log_2(1/p_i)$	分配的代码	需要的位数
B	10	1.585	11	20
A	8	1.907	10	16
C	3	3.322	000	9
D	4	2.907	001	12
E	5	2.585	01	10
合计	30	1.0		67

30个字符组成的字符串需要67位

(2) 计算该字符串的熵

其中, $X = \{x_1, \dots, x_n\}$ 是事件 $x_i (i = 1, 2, \dots, n)$ 的集合,

并满足 $\sum_{i=1}^n p(x_i) = 1$ $H(X) = -\sum_{i=1}^n p(x_i) \log_2 p(x_i)$

$\triangleright H(S)$

$$\begin{aligned} &= (8/30) \times \log_2(30/8) + (10/30) \times \log_2(30/10) + \\ &\quad (3/30) \times \log_2(30/3) + (4/30) \times \log_2(30/4) + \\ &\quad (5/30) \times \log_2(30/5) \\ &= [30 \lg 30 - (8 \times \lg 8 + 10 \times \lg 10 + 3 \times \lg 3 + 4 \times \lg 4 + 5 \lg 5)] / (30 \times \log_2 2) \\ &= (44.3136 - 24.5592) / 9.0308 = 2.1874 \text{ (Sh)} \end{aligned}$$

■ 初始化

- 根据信源符号的概率把间隔[0, 1)分成如表3-4所示的4个子间隔: [0, 0.1), [0.1, 0.5), [0.5, 0.7), [0.7, 1)。其中[x, y)的表示半开放间隔，即包含x不包含y，x称为低边界或左边界，y称为高边界或右边界

表3-4 例1的信源符号概率和初始编码间隔				
符号	00	01	10	11
概率	0.1	0.4	0.2	0.3
初始编码间隔	[0, 0.1)	[0.1, 0.5)	[0.5, 0.7)	[0.7, 1)

- 确定符号的编码范围
 $low_{i+1} = low_i + range_i \times range_low$
 $high_{i+1} = low_i + range_i \times range_high$
- 整个编码过程如图3-3所示
- 编码和译码的全过程分别见表3-5和表3-6

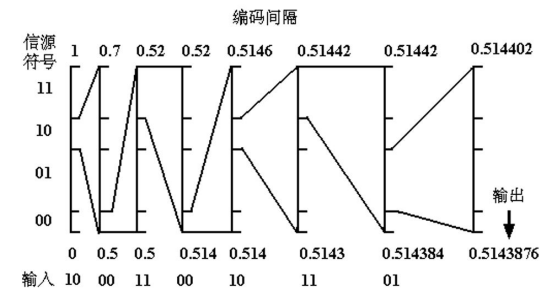


表3-5 编码过程			
步骤	输入符号	编码间隔	编码判决
1	10	[0.5, 0.7]	符号的间隔范围[0.5, 0.7)
2	00	[0.5, 0.52]	[0.5, 0.7]间隔的第一个1/10
3	11	[0.514, 0.52]	[0.5, 0.52]间隔的最后三个1/10
4	00	[0.514, 0.5146]	[0.514, 0.52]间隔的第一个1/10
5	10	[0.5143, 0.51442]	[0.514, 0.5146]间隔的第五个1/10开始，2个1/10
6	11	[0.514384, 0.51442]	[0.5143, 0.51442]间隔的最后三个1/10
7	01	[0.5143876, 0.514402]	[0.514384, 0.51442]间隔的4个1/10，从第一个1/10开始
8			从[0.5143876, 0.514402]中选择一个数(如0.5143876)作为输出

(3) 计算该字符串的平均码长

平均码长: $l = \sum_{i=1}^N l_i p(l_i)$

$$= (2 \times 8 + 2 \times 10 + 3 \times 3 + 3 \times 4 + 2 \times 5) / 30$$
$$= 2.233 \text{ 位/符号}$$

平均码长: $67/30 = 2.233 \text{ 位}$

(4) 计算编码前后的压缩比

- 编码前: 5个符号需3位, 30个字符, 需要90位
 - 编码后: 共67位
- 压缩比: $90/67 = 3/2.233 = 1.34:1$

算术编码

- 算术编码(Arithmetic Coding)
 - 给已知统计信息的符号分配代码的数据无损压缩技术
 - 基本思想是用0和1之间的一个数值范围表示输入流中的一个字符，而不是给输入流中的每个字符分别指定一个码字
 - 把整个信源表示为0到1之间的一个实数区间，其长度等于该序列的概率，再在该区间内选择一个代表性的小数，转化为二进制作为实际的编码输出
 - 实质上是为整个输入字符流分配一个“码字”，因此它的编码效率可接近于熵
- [例1]
 - 假设信源符号为{00, 01, 10, 11}，它们的概率分别为{0.1, 0.4, 0.2, 0.3}
 - 对二进制消息序列10 00 11 00 10 11 01进行算术编码

表3-6 译码过程			
步骤	间隔	译码符号	译码判决
1	[0.5, 0.7]	10	0.51439 在间隔 [0.5, 0.7)
2	[0.5, 0.52]	00	0.51439 在间隔 [0.5, 0.7) 的第 1 个 1/10
3	[0.514, 0.52]	11	0.51439 在间隔[0.5, 0.52) 的第 7 个 1/10
4	[0.514, 0.5146]	00	0.51439 在间隔[0.514, 0.52) 的第 1 个 1/10
5	[0.5143, 0.51442]	10	0.51439 在间隔[0.514, 0.5146) 的第 5 个 1/10
6	[0.514384, 0.51442]	11	0.51439 在间隔[0.5143, 0.51442) 的第 7 个 1/10
7	[0.51439, 0.5143948]	01	0.51439 在间隔[0.51439, 0.5143948) 的第 1 个 1/10
7	译码的消息: 10 00 11 00 10 11 01		

词典编码

- 词典编码是文本中的词用它在词典中表示位置的号码代替的一种无损数据压缩方法。
- 词典编码根据数据本身包含有重复编码这个特性
- 词典是指
 - (1) 用以前处理过的数据来表示编码过程中遇到的重复部分;
 - (2) 可以是任意字符的组合，编码数据过程中当遇到已经在词典中出现的“短语”时，编码器就输出这个词典中的短语的“索引号”，而不是短语本身。

例: 对一个最简单的三字符A,B,C组成的字符串ABBABABAC进行LZW编码。

表1 待编码的字符串									
位置	1	2	3	4	5	6	7	8	9
字符	A	B	B	A	B	A	B	A	C

表2 LZW的编码过程		
步骤	词典	输出
	(1) A	
	(2) B	
	(3) C	
1	(4) AB	(1)
2	(5) BB	(2)
3	(6) BA	(2)
4	(7) ABA	(4)
5	(8) ABAC	(7)
6	--	(3)

表3 LZW的译码过程			
步骤	词典	输入代码	输出
	(1) A		
	(2) B		
	(3) C		
1	--	(1)	A
2	(4) AB	(2)	B
3	(5) BB	(2)	B
4	(6) BA	(4)	AB
5	(7) ABA	(7)	ABA
6	(8) ABAC	(3)	C

五、 图像的有损压缩
失真度量

- 评估编码系统性能的一种方法是失真度量法
- 失真度量是一种在某种失真标准下一个近似值与原值的接近程度的数学量。
- 常用：均方误差（MSE），信噪比（SNR），峰值信噪比（PSNR）
- 均方误差（Mean Square Error, MSE）

MSE = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} |x(m,n) - \hat{x}(m,n)|^2

式中，x(m,n)为原始图像的像素值，\hat{x}(m,n)为解压缩后的像素值。

图像的峰值信号比 PSNR

- 峰值信噪比（Peak Signal to Noise Ratio, PSNR）

用峰值信号与噪声之比衡量, 定义为最大信号峰值的平方与信号的均方误差(MSE)之比

PSNR = 10 \log_{10} \frac{(Peak\ Signal\ Value)^2}{MSE} \quad (dB)

对 8 位二进制图像

PSNR = 10 \log_{10} \frac{255^2}{MSE} \quad (db)

图像的信噪比 SNR

➢ 信噪比（Signal to Noise Ratio, SNR）

SNR = 10 \cdot \log_{10} \left[\frac{\sum_{x=1}^{N_x} \sum_{y=1}^{N_y} f(x,y)^2}{\sum_{x=1}^{N_x} \sum_{y=1}^{N_y} (f(x,y) - \hat{f}(x,y))^2} \right]

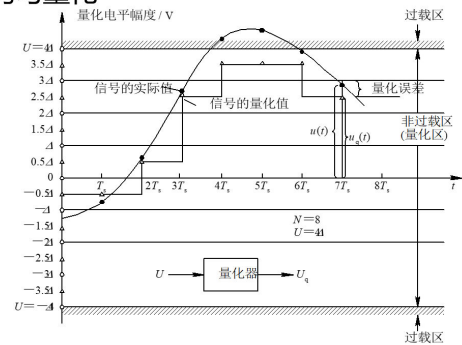
➢ PSNR与SNR的关系

PSNR(dB) = SNR(dB) + 10 \cdot \log_{10} \left[\frac{\sum_{x=1}^{N_x} \sum_{y=1}^{N_y} (255)^2}{\sum_{x=1}^{N_x} \sum_{y=1}^{N_y} (f(x,y))^2} \right]

- ◆ 例：原信号{12 16 16 12 12 8 8 12}，压缩后数据{8 12}，解压后数据{12 12 12 12 12 12 12 12}
- MSE = (4^2+4^2+(-4)^2+(-4)^2) / 8 = 8
- SNR=10*log₁₀((12^2+16^2+16^2+12^2+12^2+8^2+8^2+12^2) / (4^2+4^2+(-4)^2+(-4)^2)) = 12.79 db
- PSNR = 10*log₁₀(16^2 / 8) = 15.05 db

均匀量化

◆ 均匀量化



量化间隔计算

量化级间隔（级差） Δ = \frac{2U}{N}

- N: 量化级数（等于离散值的数目）
- U: 临界过载电压。
- 过载电压：幅度大于U（本图为大于4Δ）的电压。
- 量化区：|u(t)|≤U的区域
- 量化区内，都不会超过 Δ/2
- 量化误差
- 量化过载区内，都超过 Δ/2

均匀量化有关推论

- 均匀量化，其量化间隔 Δ = \frac{2U}{N}
- 码字数n与量化级数N的关系是N=2^n
- 在一定的量化区内，级数越多，量化间隔越小，噪声越小，信噪比越大，但编码位数越多。

非均匀量化

一维离散余弦变换

- Given an input function f(i) over one integer variable i (a piece of a vector), the 1D DCT transforms it into a new function F(u), with integer u running over the same range as i. The general definition of the transform is:

F(u) = \sqrt{\frac{2}{M}} C(u) \sum_{i=0}^{M-1} \cos \frac{(2i+1)u\pi}{2M} f(i)

- i, u = 0, 1, ..., M-1;
- The constants C(u) are determined by

C(u) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } u = 0, \\ 1 & \text{otherwise.} \end{cases}

一维逆变换

- ◆ When M = 8, 1D Inverse Discrete Cosine Transform (1D-IDCT)

\tilde{f}(i) = \sum_{u=0}^7 \frac{C(u)}{2} \cos \frac{(2i+1)u\pi}{16} F(u)

例：8 元素数据序列

- Consider a data sequence with 8 numbers
- 1D DCT

F(u) = \frac{C(u)}{2} \sum_{i=0}^7 \cos \frac{(2i+1)u\pi}{16} f(i)

- 1D IDCT

\tilde{f}(i) = \sum_{u=0}^7 \frac{C(u)}{2} \cos \frac{(2i+1)u\pi}{16} F(u)

- The constants C(u) are determined by

C(u) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } u = 0, \\ 1 & \text{otherwise.} \end{cases}

例 2: DCT 对数据序列各分量相关性的影响

- ✓ F 是输入向量 f 的线性变换 T 的结果, 其方式使得 F 的分量之间的相关性要小得多
- ✓ Example 3(textbook-p159-Ex8.1) of transform coding
- ✓ Input vector $X: f \{100 \ 100 \ 100 \ 100 \ 100 \ 100 \ 100 \ 100\}$
- ✓ Transform coding $T: \quad 1D \ DCT$

$$F(u) = \frac{C(u)}{2} \sum_{i=0}^7 \cos \frac{(2i+1)u\pi}{16} f(i)$$

- ✓ Transformed vector $Y: \quad F \{283 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0\}$

例 3: IDCT 逆变换

- ◆ Input vector $X: f$
 - $\{100 \ 100 \ 100 \ 100 \ 100 \ 100 \ 100 \ 100\}$
- ◆ Transformed vector $Y: F$
 - $\{283 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0\}$
- ◆ Invert DCT

$$\tilde{f}(i) = \sum_{u=0}^7 \frac{C(u)}{2} \cos \frac{(2i+1)u\pi}{16} F(u)$$

- ◆ Output data $X': f'$
 - $\{100 \ 100 \ 100 \ 100 \ 100 \ 100 \ 100 \ 100\}$

二维 DCT 变换

- ◆ Given an input function $f(i, j)$ over two integer variables i and j , the 2D DCT transforms it into a new function $F(u, v)$, with integer u and v running over the same range as i and j . The general definition of the transform is:

$$F(u, v) = \frac{2C(u)C(v)}{\sqrt{MN}} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \cos \frac{(2i+1) \cdot u\pi}{2M} \cdot \cos \frac{(2j+1) \cdot v\pi}{2N} \cdot f(i, j)$$

- $i, u = 0, 1 \dots \dots M-1$;
- $j, v = 0, 1 \dots \dots N-1$;
- The constants $C(u)$ and $C(v)$ are determined by:

$$C(\xi) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } \xi = 0, \\ 1 & \text{otherwise.} \end{cases}$$

二维 DCT 逆变换

- ◆ 2D Inverse Discrete Cosine Transform (2D IDCT)

$$\tilde{f}(i, j) = \sum_{u=0}^7 \sum_{v=0}^7 \frac{C(u)C(v)}{4} \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} F(u, v)$$

例题: 8×8 矩阵

- ◆ Consider an image of (8×8)
- ◆ 2D DCT

$$F(u, v) = \frac{C(u)C(v)}{4} \sum_{i=0}^7 \sum_{j=0}^7 \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} f(i, j)$$

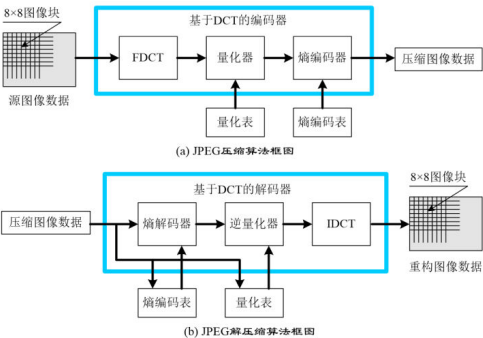
- ◆ 2D IDCT

$$\tilde{f}(i, j) = \sum_{u=0}^7 \sum_{v=0}^7 \frac{C(u)C(v)}{4} \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} F(u, v)$$

六、 JPEG 的计算步骤

- ◆ JPEG算法的计算步骤

- (0) 将图像(如RGB颜色空间)转换为YCbCr并进行二次采样
- (1) 正向离散余弦变换(FDCT)
- (2) 量化(quantization)
- (3) Z字形编码(zigzag scan)
- (4) 使用DPCM对直流系数(DC)进行编码
- (5) 用行程长度编码(RLE)对交流系数(AC)进行编码
- (6) 熵编码(entropy coding)



1、正向离散余弦变换

- 2.1. 离散余弦变换

二维离散余弦变换(2D DCT)也叫做正向离散余弦变换(2D FDCT), 用下式表示,

$$Y = AXA^T \tag{5-1}$$

2D DCT的逆变换(2D IDCT)为,

$$X = A^T Y A \tag{5-2}$$

X 是输入样本矩阵, Y 是变换后的系数矩阵, A 是 $N \times N$ 变换矩阵. A 的元素是,

$$A_{ij} = C_i \cos \frac{(2j+1)i\pi}{2N}, \text{ 其中, } C_i = \begin{cases} \sqrt{\frac{1}{N}} & (i=0), \\ \sqrt{\frac{2}{N}} & (i>0) \end{cases} \tag{5-3}$$

例题

- ◆ 考虑一个子图像块(8×8)
- ◆ 2D DCT

$$F(u, v) = \frac{C(u)C(v)}{4} \sum_{i=0}^7 \sum_{j=0}^7 \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} f(i, j)$$

- ◆ 2D IDCT

$$\tilde{f}(i, j) = \sum_{u=0}^7 \sum_{v=0}^7 \frac{C(u)C(v)}{4} \cos \frac{(2i+1)u\pi}{16} \cos \frac{(2j+1)v\pi}{16} F(u, v)$$

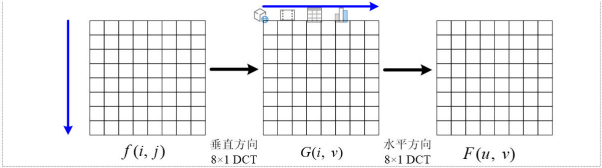
- ◆ $C(u)$ 与 $C(v)$: $C(\xi) = \begin{cases} \frac{\sqrt{2}}{2} & \text{if } \xi = 0, \\ 1 & \text{otherwise.} \end{cases}$

- ◆ $f(i, j)$ 是图像样本矩阵或样本的预测误差矩阵, $F(u, v)$ 是 $f(i, j)$ 经过 DCT 后的系数矩阵, $F(0, 0)$ 是 8×8 个像素值的平均值, 称为直流系数(DC), 其他称为交流系数(AC)

(3) 在计算二维DCT时，可用下面的计算式把二维DCT变成一维DCT，如图 5-9 所示，实际的快速计算方法可参看参考文献[4]，

$$F(u,v) = \frac{1}{2} C(u) \left[\sum_{i=0}^7 G(i,v) \cos \frac{(2i+1)u\pi}{16} \right]$$
$$G(i,v) = \frac{1}{2} C(v) \left[\sum_{j=0}^7 f(i,j) \cos \frac{(2j+1)v\pi}{16} \right]$$

(5-6)



2、量化

量化用下式计算，

$$\hat{F}(u,v) = round(\frac{F(u,v)}{Q(u,v)})$$

(5-7)

其中， $F(u,v)$ 表示 DCT 系数， $Q(u,v)$ 表示“量化矩阵”， $\hat{F}(u,v)$ 表示量化(后的)系数， $F(u,v)/Q(u,v)$ 表示两个矩阵的对应元素相除， $round$ 表示四舍五入。

关于 $Q(u,v)$

- 使用两种量化表
 - 表5-6用于亮度量化表，表5-7用于色度量化表
 - 人眼对低频分量图像比高频分量图像更敏感，因此表中的左上角的量化步距要比右下角的量化步距小
 - 表中数值对CCIR 601标准电视图像是最佳的

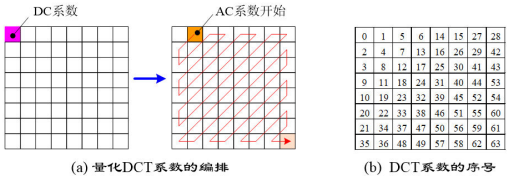
16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

3、z 字形编码

- 2.3. Z字形排列

- 重新编排量化后的系数，增加连续的0值系数数目
- 排列方法：按Z字形排列，如下图(a)
- DCT系数序号见图5-11(b)，序号小的位置表示频率较低，用 $zz(0), zz(1), \dots, zz(63)$ 表示
- 8×8 的矩阵变成 1×64 的矢量



4、熵编码

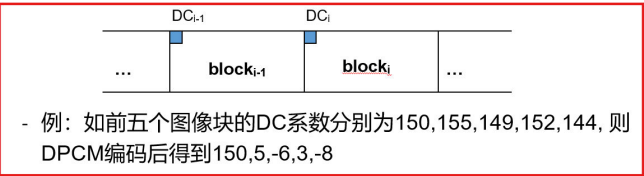
- 2.4. 熵编码

- 目的：压缩采用差分脉冲调制编码(DPCM)编码后的DC系数差值和RLE编码后的AC系数
- 方法：使用霍夫曼编码器，用查表方法编码，霍夫曼码表可事先定义
- 熵编码过程：
 - DC系数编码
 - DPCM编码
 - 生成中间符号
 - 符号编码
 - AC系数编码
 - 生成中间符号
 - 符号编码

4.1 DC 系数编码（差分脉冲编码）

- 直流系数(DC)的编码
 - DC系数的特点： 8×8 图像块经过DCT变换之后得到的DC直流系数有两个特点，一是系数的数值比较大，二是相邻 8×8 图像块的DC系数值变化不大。
 - JPEG算法使用了差分脉冲调制编码(DPCM)技术，对相邻图像块之间量化DC系数的差值(Delta)进行编码。

$$\Delta DC = DC(0,0)_i - DC(0,0)_{i-1}$$



4.2 AC 系数的编码（行程长度编码 RLE）之前在无损压缩中有提及，但不涉及计算

◆ 交流系数(AC)的编码

- AC系数的特点： 1×64 矢量中包含有许多“0”系数，并且许多“0”是连续的。
- JPEG使用非常简单和直观的行程长度编码(RLE)对它们进行编码。
- JPEG使用了1个字节的高4位来表示连续“0”的个数，而使用它的低4位来表示编码下一个非“0”系数所需要的位数，跟在它后面的是量化AC系数的数值。

例： $\hat{F}(u,v)$ 经Z字扫描后为(32,6,-1,-1,0,-1,0,0,-1,0,0,1,0,0,...0)
RLE后: (0,6)(0,-1)(0,-1)(1,-1)(3,-1)(2,1)(0,0)

4.3 熵编码（霍夫曼编码）

◆ 熵(Entropy)编码

- 使用熵编码的原因：对DPCM编码后的直流DC系数和RLE编码后的交流AC系数作进一步的压缩。
- 在JPEG有损压缩算法中，使用霍夫曼编码器来减少熵，霍夫曼编码器使用很简单的查表(lookup table)方法进行编码。

压缩数据符号时，霍夫曼编码器对出现频度比较高的符号分配比较短的代码，而对出现频度较低的符号分配比较长的代码。这种可变长度的霍夫曼码表可以事先进行定义。

- DC码表符号举例

如果DC的值(Value)为5或-5，符号SSS用于表达实际值所需要的位数，实际位数就等于3。
表示为：中间符号(3, 5),或(3, -5)

Value	SSS
0	0
-1, 1	1
-3,-2, 2,3	2
-7,..-4, 4,..7	3

七、 图像检索

均值哈希 aHash

基本思想：比较灰度图每个像素与平均值来实现

步骤：

- 1. 缩小图片：为了保留结构去掉细节，去除大小、横纵比的差异，把图片统一缩小到 8*8(像素)，共 64 个像素。
- 2. 转化为灰度图：把缩放后的图片转化为 256 阶的灰度图。
- 3. 计算平均值： 计算进行灰度处理后图片的所有像素点的平均值。
- 4. 比较像素灰度值： 遍历 64 个像素，如果大于平均值记录为 1， 否则为 0。
- 5. 得到信息指纹： 组合 64 个 bit 位，顺序随意保持一致性即可。
- 6. 对比指纹： 计算两幅图片的汉明距离，汉明距离越大则说明图片越不一致，反之，汉明距离越小则说明图片越相似。

结论：

- 1. 当距离为 0 时，说明完全相同；
- 2. 通常认为距离 > 10 就是两张完全不同的图像；
- 3. 如果汉明距离小于5，则表示有些不同，但比较相近。

优点：

- 1. 图像放大或缩小，或改变纵横比，Hash值不会改变；
- 2. 增加或减少亮度或对比度，或改变颜色，对hash值都不会有太大的影响；
- 3. 计算速度快。

缺点：

丢失了高频信息，丢失了细节

感知哈希 pHash

基本思想：使用离散余弦变换(DCT)来获取图像的低频成分

步骤：

- 1. 缩小尺寸： pHash从小图像开始，但图像大于8*8， 32*32是最好的。目的是简化DCT的计算，而不是减小频率；
- 2. 转化为灰度图像： 进一步简化计算量；
- 3. 计算DCT： 计算图像的DCT变换，得到32*32的DCT系数矩阵；
- 4. 缩小DCT： 虽然DCT的结果是32*32大小的矩阵，但我们只要保留左上角的8*8的矩阵，这部分呈现了图像中的最低频率；
- 5. 计算平均值： 如同均值哈希一样，计算DCT的均值；
- 6. 计算hash值： 如同均值哈希一样，根据8*8的DCT矩阵，设置0或1的64位的hash值，大于等于DCT均值的设为“1”， 小于DCT均值的设为“0”。组合在一起，就构成了一个64位的整数，这就是这张图像的指纹。

优点：

- 1. 只要图像的整体结构保持不变，hash结果值就不变；
- 2. 能够避免伽马校正或颜色直方图被调整带来的影响

缺点：

计算速度较aHash慢

dHash 差异哈希

基本思想：基于渐变实现

步骤：

- 1. 缩小图像： 缩小到 9(列)*8(行) 的大小，共 72 个像素点；
- 2. 转化为灰度图： 把缩放后的图片转化为 256 阶的灰度图；
- 3. 计算差异值： dHash 算法工作在相邻像素之间，这样每行 9 个像素之间产生了 8 个不同的差异， 8 行*8，则产生了 64 个差异值；
- 4. 获得指纹： 如果左边像素的灰度值比右边高，则记录为 1， 否则为 0；
- 5. 对比指纹： 同平均哈希算法。

优点：

- 1. 相比 pHash， dHash 的速度要快得多；
- 2. 相比 aHash， dHash 在效率几乎相同的情况下的效果要更好。

mAP 的计算

假设返回K个检索结果，其中C个是相似（相关）的图像，检索数据集实际有N个相似图像。

- ◆ 准确率 (Precision) : $Precision@K = C / K$
- ◆ 召回率 (Recall, 又叫查全率) : $Recall@K = C / N$
- ◆ 平均检索精度 (Average Precision, AP) 代表不同召回率下准确率的平均值，是一种同时考虑召回率和准确率的评价指标
- ◆ $AP = \frac{1}{C} \sum_{k \in S} P@k$, 其中C为相关的图像数目，k为相似的图像所在的排序位置，S为所有相似图像排序位置的集合
- ◆ 平均检索精度均值 (mAP) 是查询多张图像的检索结果AP的平均值。如Q张查询图像的检索精度AP分别为AP₁,AP₂,...,AP_Q, 那么平均检索精度均值mAP = $\frac{1}{Q} \sum_{i=1}^Q AP_i$
- ◆ 平均检索精度均值 (mean Average Precision, mAP) 在利用mAP的评估的时候，需要知道：
 - 1. 每个Query有多少个相似的图像，计算AP时需要考虑所有相似的图像；
 - 2. 排序结果中这些相似图像的位置；
 - 3. 相似的定义（一般事先会定义）。

例题：

- ◆ 假设两个查询图，查询图1有4张相关图，查询图2有5张相关图。某系统对于主题1的4张相关图分别的排位是：1， 2， 4， 7。对于查询图2，只检索出3张图，相关排位是1， 3， 5。请计算MAP。

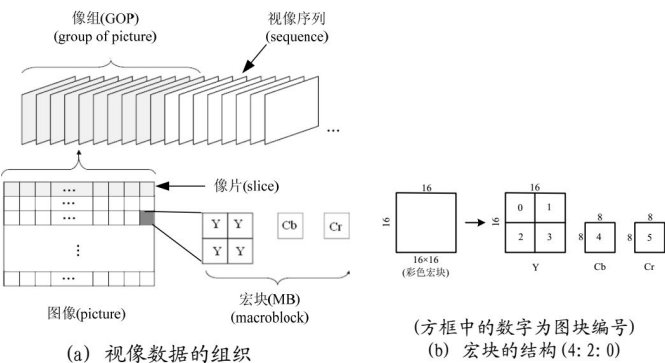
- ◆ 查询图1：平均准确率为 $(1/1+2/2+3/4+4/7)/4 = 0.83$

- ◆ 查询图2：平均准确率为 $(1/1+2/3+3/5+0+0)/5 = 0.45$
MAP=(0.83+0.45)/2=0.64

八、 MPEG

数据结构

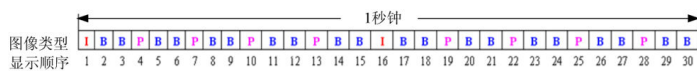
- ◆ 把视像片段看成由一系列静态图像(picture)组成的视像序列(sequence)；
- ◆ 把视像序列分成许多像组(group of picture, GOP)；
- ◆ 把像组中的每一帧图像分成许多像片(slice)，每个像片由16行组成；
- ◆ 把像片分成16×16像素的宏块(macroblock, MB)；
- ◆ 把宏块分成若干个8×8像素的图块(block)，见图1(a)；
- ◆ 使用子采样格式为4:2:0时，一个宏块由4个亮度(Y)图块和两个色度图块(Cb和Cr)组成，见图1(b)。



- 帧内图像 I 出现的频率和位置。通常为 2 Hz
- 在两帧图像 I 之间或在图像 I 和 P 之间选择图像 B 的数目
- 图像 I、P 和 B 的数目主要是根据节目的内容来确定。

◆ 一个I、P和B的典型编排顺序见图3

- 在视像解码时，因B 需 I和 P做参考，故在解码之前需重新组织帧图像数据流的输入顺序，其方案见图4



帧图像的
显示顺序

视像流的
传输顺序

▶ 画面的显示顺序是：

B	B		B	B	P	B	B	P	B	
0	1	2	3	4	5	6	7	8	9	10

▶ 画面的编码顺序是：

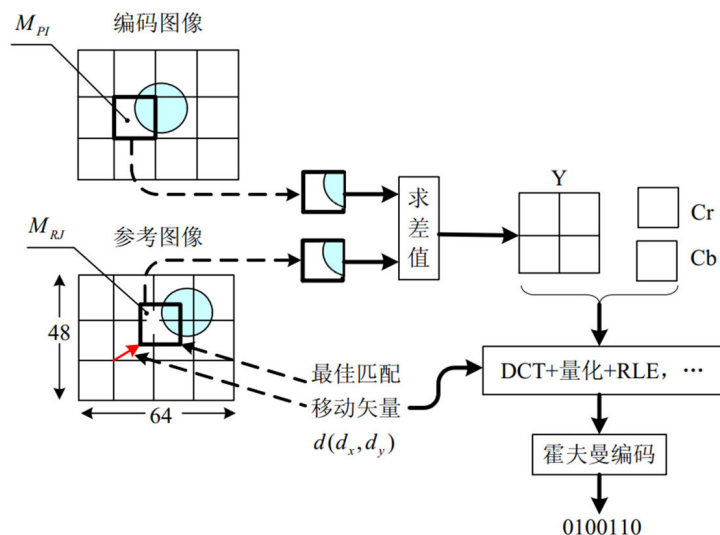
	B	B	P	B	B	P	B	B		B
2	0	1	5	3	4	8	6	7	10	9

- ◆ 如果视像是用RGB空间表示的视像，则首先把它转换成YCrCb空间表示的视像

- ◆ 每个图像平面分成 8×8 像素的图块，对每个图块进行离散余弦变换(DCT)，变换后产生的交流分量系数经过量化之后按照Zig-zag的形状排序。DCT得到的直流分量系数经过量化之后用差分脉冲编码(DPCM)，交流分量系数用行程长度编码RLE，然后再用霍夫曼(Huffman)编码或者用算术编码

(1)求解差值的方法(见图8)

- ◆ 假设编码宏块 M_{PI} 是参考宏块 M_{RJ} 的**最佳匹配块**，它们的**差值**就是这两个宏块中相应的**像素值之差**。
- ◆ 对所求得的差值进行**彩色空间转换**，然后使用4:1:1或4:2:0格式采样。对采样得到的Y, Cr和Cb分量值，仿照JPEG压缩算法对差值进行编码。
- ◆ 对计算出的移动矢量进行DCT变换和霍夫曼编码

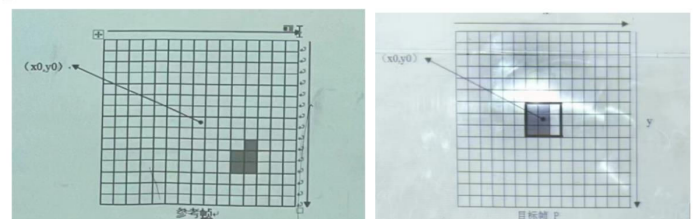


(2)求解运动向量

- ◆ 在求两个宏块差值之前，需要找出预测编码图像中的编码宏块 M_{ip} 相对于参考图像中的参考宏块 M_{rp} 所移动的距离和方向，即运动向量。
- ◆ 求解的目标是找到一个向量 (i, j) 作为运动向量 $MV=(u, v)$ ，使 $MAD(i, j)$ 取最小值。 MAD (Mean Absolute Difference)用于表示两个宏块的差

$$MAD(i, j) = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} |C(x+k, y+l) - R(x+i+k, y+j+l)|$$

目标帧为P帧，宏块大小是3x3，运动向量 $MV((v_x, v_y))$ 点 $(v \in [-p, p])$, $(p = 5)$ 。目标P帧中宏块中心坐标是 (x_0, y_0) ，包含6个黑色像素(一个小单元格表示一个像素)，每个黑色像素值为50，其余值为100。



运动向量 $(4, 3)$

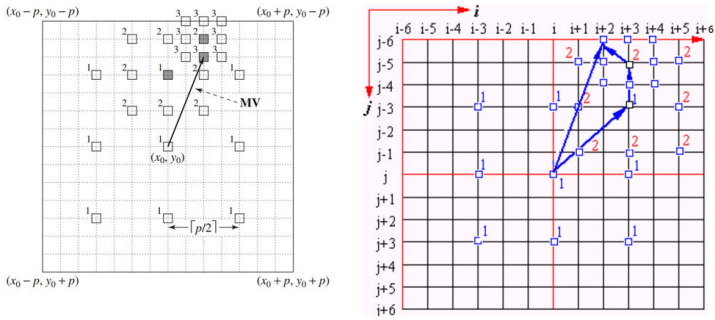
$$MAD(i, j) = \frac{1}{N^2} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} |C(x+k, y+l) - R(x+i+k, y+j+l)| = (100 - 50)/9 = 50/9,$$

(3) 搜索算法

- ◆ 顺序搜索 (full search)
 - 将参考帧中整个 $(2p+1) * (2p+1)$ 的窗口中的每一个宏块逐个像素与目标帧 中的宏块进行比较。
 - 获取一个宏块的运动向量的复杂度为 $(2p+1) * (2p+1) * N^2 * 3$ ，计算复杂度为 $O(p^2 N^2)$

◆ 2D对数搜索 (logarithmic search)

基本思想是在不同大小的搜索范围内进行搜索，每次迭代搜索范围都在缩小，直到找到最佳的运动向量。搜索次数减少，容易漏掉小的移动，计算复杂度为 $O(\log p \cdot N^2)$



分层搜索

◆ 分层搜索

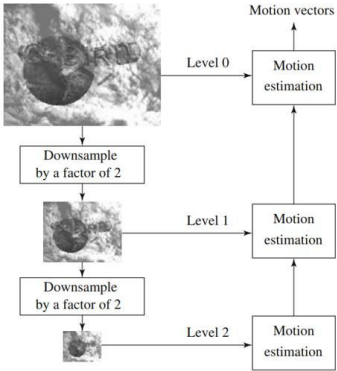
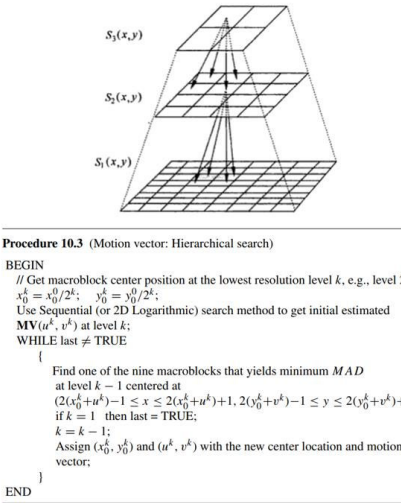


图11 分层搜索法



双向预测图像 B 的压缩算法

利用差值取平均

◆ 双向预测图像 B 不传播编码误差。

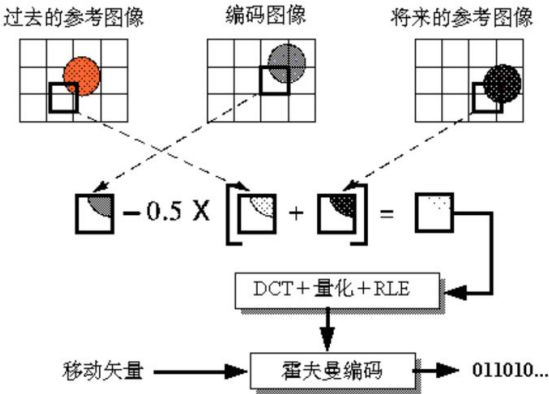


图12 双向预测图像B的压缩编码算法框图

每秒计算量的计算

(3)假设帧率为 25fps，求每秒的计算量是多少？。

P218

$$\begin{aligned} OPS_per_second &= (8 \cdot (\lceil \log_2 p \rceil + 1) + 1) \cdot N^2 \cdot 3 \cdot \frac{720 \times 480}{N \cdot N} \cdot 25 \\ &= (8 \cdot (\lceil \log_2 5 \rceil + 9) + 1) \times 3^2 \times 3 \times \frac{720 \times 480}{3 \times 3} \times 25 \\ &\approx 8.5536 \times 10^8 \end{aligned}$$